



UNIVERSIDAD NACIONAL EXPERIMENTAL DEL TÁCHIRA (UNET)

De la implementación física a la simulación

Migración del robot futbolista *FootBot* a ROS 2 + Gazebo

Chacón Gómez, José Daniel

Proyecto FootBot / Auto Soccer Bot · Junio de 2026

- 1 El punto de partida: el robot real
- 2 ¿Por qué migrar a simulación?
- 3 Del robot físico a ROS 2 + Gazebo
- 4 Arquitectura del sistema
- 5 Fases del trabajo
- 6 Comportamientos autónomos
- 7 Percepción y *datasets*
- 8 Resultados y trabajo futuro

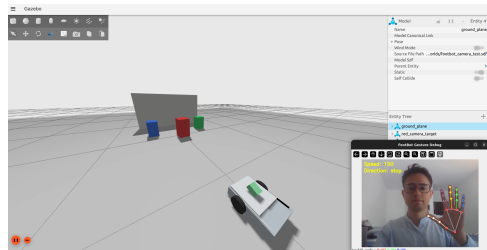
El punto de partida: el robot real

FootBot / Auto Soccer Bot: robot futbolista de bajo costo con ESP32-CAM.

Operaba en dos modos:

- **Manual:** control por gestos de la mano (MediaPipe) desde el portátil.
- **Automático:** el ESP32-CAM transmite vídeo; el portátil detecta el balón (HSV + YOLO) y una FSM lo sigue.

Percepción y decisión en el portátil; comandos por HTTP /move.



Control por gestos con MediaPipe.

¿Por qué migrar a simulación?

Límites del banco físico

- Hardware frágil y costoso
- Dependencia de la iluminación
- Un solo robot, iteración lenta

Ventajas de la simulación

- Reproducible y determinista
- Segura y sin costo por iterar
- Escenarios en paralelo
- Base para comportamientos avanzados

Principio

No se desecha lo anterior: el robot ESP32 queda como [referencia](#) y su protocolo HTTP `/move` se conserva mediante un puente.

Del robot físico al entorno simulado

Robot físico (ESP32)	Simulación (ROS 2 + Gazebo)
Tracción diferencial + L298N	Modelo URDF/Xacro + plugin DiffDrive
Cámara ESP32-CAM (MJPEG)	Cámara de Gazebo → /camera/image_raw
API HTTP /move	footbot_bridge → /cmd_vel
Visión en el portátil (HSV/YOLO)	footbot_perception / footbot_soccer_vision
FSM de seguimiento	footbot_soccer_behavior (FSM)

Estrategia *simulation-first*: reconstruir cada capacidad como paquetes de ROS 2, reutilizando los conceptos del robot real.

Arquitectura del sistema de simulación

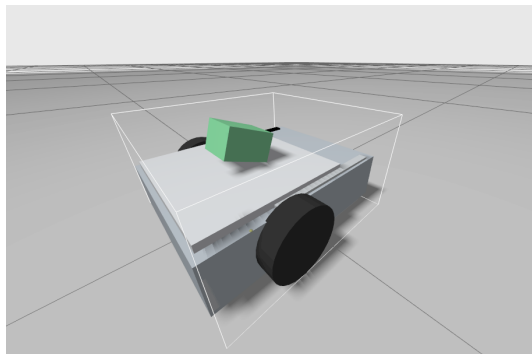
Stack

- ROS 2 Humble + Gazebo Fortress
- Integración `ros_gz` (puente ROS ↔ Gazebo)

Flujo de datos

- Gazebo → `ros_gz_bridge` → topics
- Percepción → estado → FSM → `/cmd_vel`

Regla de oro: un solo nodo dueño de `/cmd_vel`.



FootBot en Gazebo con su cámara simulada.

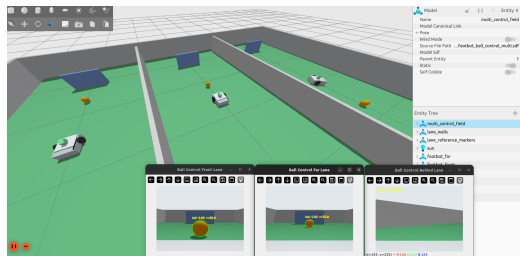
Fases del trabajo realizado

- ❶ **Fundamentos:** modelo del robot (URDF/Xacro), mundos y arranque en Gazebo.
- ❷ **Puentes de control:** `/cmd_vel`, puente HTTP `/move` (ESP32) y gestos nativos en ROS.
- ❸ **Percepción simulada:** cámara del robot, detección HSV del balón y visión YOLO.
- ❹ **Comportamientos autónomos:** *Control de balón y Llegar a la portería*, deterministas por FSM.
- ❺ **Datasets y entrenamiento:** captura, etiquetado, aumento y entrenamiento de YOLO.

Comportamiento 1: Control de balón

Encontrar, aproximarse, contactar y **mantener** el balón en una zona de control frontal.

- Detección HSV → BallState
- FSM: buscar, alinear, aproximar, contactar, controlar, recuperar, parada segura
- Las *skills* solo producen comandos Twist acotados; la FSM decide las transiciones.



Depuración: balón detectado y estado.

Control de balón: ¿cómo lo mantiene centrado?

Idea: el robot **solo ve el balón en la imagen**; “centrado” = el balón en el centro del cuadro. Mide cuánto se desvía y gira para corregirlo.

1. Mide la desviación como un ángulo θ (si está centrado, $\theta = 0$):

$$\theta = \arctan\left(\frac{u - W/2}{f}\right)$$

2. Gira proporcional al error (más descentrado $\Rightarrow$ corrige más rápido):

$$\omega = -K_p \theta \quad (\text{el signo } - \text{ gira hacia el balón})$$

3. Lo da por “controlado” solo si sigue centrado y cerca un tiempo:

$$|\theta| \leq \theta_{\text{tol}} \quad \text{y} \quad d \leq d_{\text{ctrl}} \quad \text{durante} \geq T_{\text{hold}}$$

Comportamiento 2: Llegar a la portería

Empujar el balón hacia una portería visible, **guiado solo por percepción** (sin usar poses reales de Gazebo).

- Detector YOLO de ball + goal $\rightarrow$ BallGoalState.
- FSM: buscar balón, controlarlo, buscar/alinear portería, driblar y marcar.
- **Memoria temporal de la portería** cuando desaparece a corta distancia.
- COMMIT_TO_GOAL: empuje final lento, centrado en el balón.
- Un **árbitro de simulación** detecta el gol y detiene el episodio.

Detección

- Cámara → /camera/image_raw
- HSV (balón) + YOLO (balón, portería, oponente)

Tubería de datos (YOLO)

- Captura → etiquetado en Label Studio
- Aumento → validación → entrenamiento



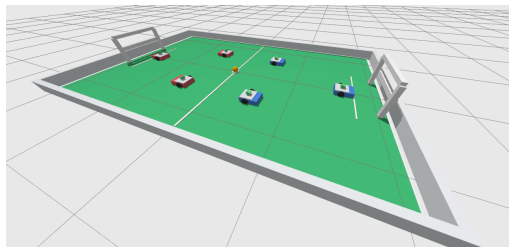
Etiquetado de objetos en Label Studio.

Resultados y estado actual

Implementado y funcionando:

- Robot, mundos y movimiento por `/cmd_vel`
- Puente HTTP `/move` y gestos en ROS
- Control de balón y Llegar a la portería
- Herramientas de *dataset* y entrenamiento YOLO

Verificación: la suite de `footbot_soccer_behavior` pasa (41 pruebas, 0 fallos).



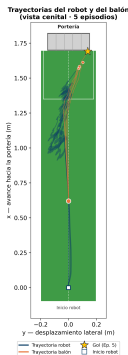
Campo de fútbol completo.

Resultado clave: el remate a corta distancia

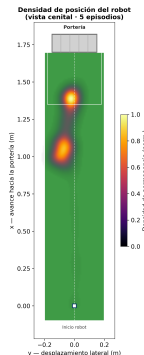
Tanda de 5 episodios de *Llegar a la portería*:

- Controla el balón rápido (≈ 9 s) y estable.
- Pero solo 1 de 5 marcó (20 %); ese tardó ≈ 269 s.
- Robot y balón se estancan frente a la portería.

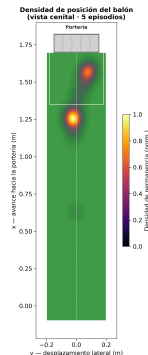
Causa: muy cerca, la portería no cabe en el cuadro y YOLO no la detecta de forma continua $\Rightarrow$ la FSM pierde la referencia y reincide en RECOVER BALL (≈ 33 % del tiempo). El respaldo COMMIT_TO_GOAL no se activó en ningún episodio.



(a) Trayectorias



(b) Densidad robot



(c) Densidad balón — vista cenital, portería arriba.

Trabajo futuro

- **Portería a corta distancia (prioritario):** *dataset* de portería cercana/parcial y reentrenar YOLO, detectar postes/línea de meta, cámara de mayor FOV, o activar el *modo de remate* COMMIT_TO_GOAL guiado por la última pose + odometría.
- **Oponentes:** detectar y disputar el balón sin desestabilizar la base.
- **Juego en equipo:** tres robots por bando, con roles y *namespaces* por robot.
- **Investigación:** árboles de comportamiento, planificación y, con métricas estables, RL/MARL.

Principio rector

Mantener la base **determinista** y medible; añadir táctica solo cuando aporte y sea depurable.

- La migración convirtió un banco físico frágil en un entorno reproducible, seguro y escalable.
- Sin desechar lo previo: el robot ESP32 sigue como referencia.
- Dos comportamientos de fútbol deterministas sobre un patrón claro: percepción → estado → FSM → /cmd_vel.
- Base sólida para oponentes, juego en equipo y experimentos tácticos.

¡Gracias!

Preguntas y comentarios

Chacón Gómez, José Daniel · UNET · Proyecto FootBot